STM32_0

실험 전 보고서

STM32_0 · LED와 Button

실험 주제 : STM32 GPIO Polling을 이용한 LED 제어 및 버튼 입력 처리

실험 대상 : STM32 NUCLEO-G474RE 개발보드, On-board LED (PA5), User Button (PC13), 외부 LED (PB0 또는 PB1), GPIO 입출력 기능

작성일 : 2026년 10월 11일

검증 환경 : VS Code STM32

성명 : 권진혁

학과 : 전자전기컴퓨터공학부

학번 : 2023440007

1. 실험목적

본 실험에서는 STM32 마이크로컨트롤러의 GPIO를 이용하여 LED를 제어하고 버튼 입력을 읽는 방법을 학습한다. 특히, Polling 방식으로 버튼의 상태를 지속적으로 확인하고, 버튼 입력에 따라 LED의 점멸 주기를 변경하는 프로그램을 구현함으로써 임베디드 시스템의 기본적인 입출력 제어 과정을 이해한다. 실험 목표는 다음과 같다.

1) GPIO 입력과 출력의 동작 원리를 이해
2) Polling 방식으로 버튼 상태 숙지
3) LED ON/OFF 제어를 구현한다.
4) 버튼 입력에 따라 LED 점멸 주기를 변경하는 프로그램 작성
5) 내부 LED와 외부 LED를 동시에 제어

2. 관련이론

· GPIO (General Purpose Input Output)

GPIO는 마이크로컨트롤러의 범용 입출력 포트이다. 입력 모드에서는 외부 장치의 상태를 읽고, 출력 모드에서는 외부 장치를 제어할 수 있다. 본 실험에서는 'PC13 : 사용자 버튼 입력', 'PA5 : 보드 내장 LED 출력', 'PB0, PB1 : 외부 출력 핀'을 사용한다.

· Polling 방식

Polling은 CPU가 반복문 내에서 입력 상태를 지속적으로 검사하는 방식이다.

```
while(1)
{
    버튼 상태 확인
    LED 제어
}
```

장점 : 구현 간단, 직관적 구조, 쉬운 디버깅
단점 : CPU 자원 지속적 사용, 인터럽트 방식보다 비효율적

· 디바운싱(Debouncing)

기계식 버튼은 한 번 눌러도 접점이 순간적으로 여러 번 붙고 떨어지는 현상(채터링)이 발생한다. 따라서 버튼을 누른 직후 짧은 지연시간을 두거나 이전 상태를 저장하여 한 번만 입력되도록 처리해야 한다.

```
static uint8_t prev = 0;

if(now && !prev)
{
    // 버튼이 처음 눌린 순간만 인식
}
```

· STM32 GPIO 관련 함수

입력 읽기 : 지정한 GPIO 핀의 상태를 읽는다. 'HAL_GPIO_ReadPin()'

출력 설정 : 지정한 핀을 HIGH or LOW로 설정. 'HAL_GPIO_WritePin()'

출력 토글 : LED ON/OFF 상태를 반전. 'HAL_GPIO_TogglePin()'

지연 함수 : 설정된 시간(ms) 동안 프로그램 실행 정지 'HAL_Delay()'

3. 실험 장비

1) STM32 NUCLEO-G474RE
2) USB 케이블
3) PC (Visual Studio Code)
4) 브레드보드
5) LED
6) 220Ω 저항
7) 점퍼선

4. 실험 내용

· **예제 실습** : 내장 LED(PA5)를 1초 간격으로 점멸시킨다.

```
void polling_routine(void)
{
    HAL_GPIO_TogglePin(
        LED_ece2_GPIO_Port,
        LED_ece2_Pin);

    HAL_Delay(1000);
}
```

· **과제 1** : 버튼(PC13)을 누를 때마다 LED 점멸 주기를 변경한다.

· **과제 2** : 과제 1에서 구현한 기능을 외부 LED(PB0 또는 PB1)에 대해서도 적용한다. 내장 LED와 외부 LED가 동일한 주기로 점멸하도록 구현한다.

5. 실험 전 예상 코드

```
void polling_routine(void)
{
    static uint32_t delay_time = 1000;
    static uint8_t prev = 0;

    uint8_t now =
        HAL_GPIO_ReadPin(
            BUTTON_ece2_GPIO_Port,
            BUTTON_ece2_Pin);

    if(now && !prev)
    {
        if(delay_time == 1000)
            delay_time = 2000;
        else if(delay_time == 2000)
            delay_time = 4000;
        else if(delay_time == 4000)
            delay_time = 8000;
        else
            delay_time = 1000;
    }

    prev = now;

    HAL_GPIO_TogglePin(
        LED_ece2_GPIO_Port,
        LED_ece2_Pin);

    HAL_Delay(delay_time);
}
```

본 실험에서는 Polling 방식을 사용하므로 polling_routine() 함수 내에서 버튼 상태를 계속 확인한다.

과제 2의 경우 외부 LED도 함께 토글한다.

HAL_GPIO_TogglePin(
GPIO_OUT1_ece2_GPIO_Port,
GPIO_OUT1_ece2_Pin);

6. 실험 방법

1) 프로젝트를 VS Code에서 실행한다.

2) ece2_474.c 파일의 'polling_routine()' 함수를 수정한다.

3) 프로젝트를 Build한다.

4) STM32 보드를 연결한다.

5) F5를 눌러 프로그램을 다운로드한다.

6) 보드의 버튼을 눌러 LED 점멸 주기가 변화하는지 확인한다.

7) 외부 LED를 연결한 뒤 동일한 동작을 확인한다.

7. 예상 결과

· **초기 상태** : LED - 1초 주기 점멸

· **버튼 1회 입력** : LED - 2초 주기 점멸

· **버튼 2회 입력** : LED - 4초 주기 점멸

· **버튼 3회 입력** : LED - 8초 주기 점멸

· **버튼 4회 입력** : LED - 1초 주기 점멸

외부 LED 역시 동일한 점멸 주기를 보일 것으로 예상된다.

8. 고찰

본 실험을 통해 GPIO 입력과 출력 제어의 기본 원리를 이해할 수 있다. 또한 Polling 방식의 특징과 한계를 확인할 수 있으며, 버튼 입력 시 발생하는 채터링 현상을 처리하기 위한 디바운싱 기법의 필요성을 확인할 수 있다. 향후 진행될 인터럽트 실습에서는 Polling보다 효율적인 입력 처리 방법을 학습할 예정이다.

9. 참고문헌

- STM32 NUCLEO-G474RE User Manual
- STM32 HAL Driver Documentation
- 전자전기컴퓨터설계실험Ⅱ 교안

10. Git-hub 주소

https://github.com/UnnamedDays/ECE-II-Project